Laboratory Report - Logger and Parentheses Validator

Student: Bahna Sebastian | **Course:** Object-Oriented Programming (Lab 1) **Teacher / Lab Designer:** Dominic Flocea | **Date:** October 2026
Repository: https://github.com/bsebb/OOP | **Language:** C++ (C++17)

1. Program Structure

What is the responsibility of `Logger`?
The `Logger` class formats diagnostic events, filters messages by severity, and prints text to the terminal. It keeps output mechanics separate from the core program logic. It stores shared settings (`level`, `format`) and an individual module tag (`name`).

What is the responsibility of `ParenthesesValidator`?
The `ParenthesesValidator` class checks whether parentheses in a string are balanced and correctly ordered. It tracks opening brackets using a stack. During the scan, it reports parser milestones: open brackets found, pairs matched, invalid closing brackets, syntax errors, and the final boolean verdict.

How do the two classes collaborate?
The two classes collaborate through composition. `ParenthesesValidator` owns a `Logger` instance named `"Parentheses"` and calls its logging methods (`trace`, `info`, `warn`, `error`, `log`). The validator does not inherit from `Logger`. Instead, it treats the logger as a helper service.

2. Classes and Objects

What is a class in your implementation?
A class is a custom blueprint that defines member variables for state and functions for behavior. In this program: - `Logger` bundles state (`name`, `level`, `format`) with functions (`log()`, `setLevel()`, `setFormat()`). - `ParenthesesValidator` bundles its private logger instance with the `isValid()` parser.

Give an example of an object created from a class.
Inside the `ParenthesesValidator` constructor, we create a concrete `Logger` object:

```
class ParenthesesValidator {
private:
    Logger logger;
public:
    ParenthesesValidator() : logger("Parentheses") {}
};
```

Here, `logger` is a real instance allocated in memory with its own identity (`name = "Parentheses"`). In `runTests()`, `ParenthesesValidator validator;` creates a validator object.

What state does each object store?
- **`Logger` object:** Stores its instance attribute `name` (a string with the module label).
- **`ParenthesesValidator` object:** Stores its internal `logger` object. While `isValid()` runs, it also creates a local stack (`std::vector<int> stack`) to hold unmatched opening bracket indices.

3. Static and Instance Members

Which members of `Logger` are static, and why?
- `static LogLevel level`: the minimum threshold for printing logs.
- `static std::string format`: the template string used to build log lines.
- `static void setLevel(LogLevel)` and `static void setFormat(std::string)`: updates the shared settings.
Why: Both `level` and `format` represent application-wide settings. Since they are static, one change updates every logger in the program right away. We do not need to loop through and update individual objects.

Which member belongs to an individual logger instance?
`std::string name` belongs to each instance. It lets different subsystems (like `"Parentheses"`, `"Database"`, or `"Network"`) identify their own messages while using the same shared format.

What happens when the logging level is changed? Does it affect every logger instance? Explain why. Include snippets.
Changing the logging level affects every `Logger` instance right away. Static members live in the program's static storage segment rather than inside any single object. Only one copy exists in memory. When we call `Logger::setLevel()`, we update that single shared variable:

```
Logger::setLevel(LogLevel::INFO); // TRACE messages stay hidden; only INFO, WARN, ERROR print
validator.isValid("()");
Logger::setLevel(LogLevel::TRACE); // Now TRACE messages print across all logger instances
```

Inside `Logger::log()`:

```
if (static_cast<int>(msgLevel) < static_cast<int>(level)) { return; }
```

4. Methods and Responsibilities

What is the responsibility of the `log` method?
The `log()` method does three jobs: 1. **Filter:** It drops any message whose severity is below `Logger::level`. 2. **Format:** It replaces the template tokens (`%t`, `%l`, `%n`, `%m`, `%r`) with real runtime values. 3. **Print:** It writes the completed string to `std::cout`.

Why does `ParenthesesValidator` use a logger instead of implementing logging itself?
This keeps each class focused on one task. `ParenthesesValidator` only handles bracket syntax. It does not need to know about clocks, string formatting, or console output. If we change how messages print later, we only edit `Logger`.

Identify one example of a method call between objects or classes in your program.
Inside `ParenthesesValidator::isValid`:

```
logger.trace("Found opening parenthesis at index " + std::to_string(i));
```

Here, the `ParenthesesValidator` calls the `trace` method on its private `logger` object.

5. Testing and Correctness

Give one valid and one invalid input for the parentheses validator.
- **Valid input:** `"(()((()()))())"` returns `true`. Each opening bracket matches a closing bracket in the right order.
- **Invalid input:** `")((((()))"` returns `false`. The first character is a closing bracket with no opening bracket before it.

Explain how the program determines whether the input is valid.
The program scans the string from left to right using a stack: 1. When it reads `'('`: it pushes the index `i` onto `stack` and logs a `TRACE` event. 2. When it reads `')'`: - If `stack` is empty: no opening match exists. It logs `WARN` and `ERROR` events and returns `false` right away. - If `stack` has elements: it pops the top index and logs an `INFO` event for the matched pair. 3. After the loop: it checks `if (!stack.empty())`. If unclosed opening brackets remain, it logs an `ERROR` with the open count and returns `false`. 4. If the stack is empty, it logs an `INFO` event and returns `true`.

Describe one test that helped you find or prevent an error.
Testing the input `"("`. An algorithm that only checks for bad closing brackets would wrongly mark `"("` as valid, since no `)` ever causes an error. This test proved that the stack must be checked after the loop finishes.

6. Reflection

What was the most important OOP concept you applied in this laboratory?
The distinction between static and instance scope. Storing `name` on each object gives each component its own identity, while storing `level` and `format` on the class gives us a central control point. Combining that with composition showed how to delegate work cleanly instead of cramming everything into one large class.

If you had more time, what would you improve in your design, and why?

1. **Configurable Log Sinks:** `Logger` currently writes straight to `std::cout`. I would add an abstract sink interface so users can route logs to disk files or memory buffers without editing `Logger`.
2. **Support for Other Brackets:** I would expand `ParenthesesValidator` to check curly braces `{}` and square brackets `[]` using a small lookup table for matching pairs.